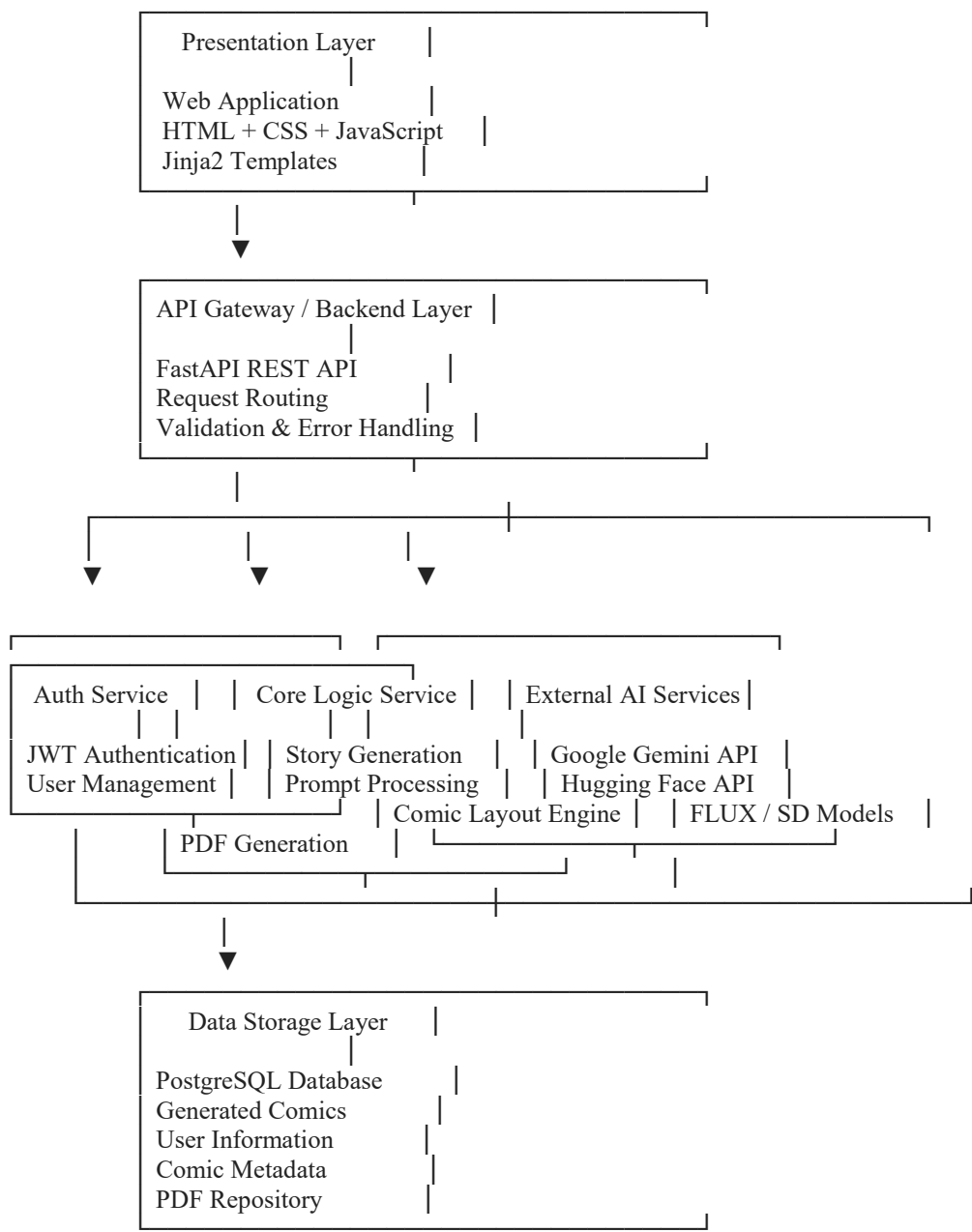


SOLUTION ARCHITECTURE

Project Name

ComicCraft – AI Powered Comic Story Generator

Solution Architecture Diagram



Architecture Overview

ComicCraft follows a layered architecture that separates the user interface, backend services, AI integrations, and data storage components. This design improves scalability, maintainability, security, and performance.

The system receives user prompts through a web interface, processes them using AI-powered services, generates comic stories and illustrations, assembles the comic layout, and stores the results for download and future access.

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides the user interface for entering prompts, selecting comic styles, previewing comics, and downloading PDFs.	HTML5, CSS3, JavaScript, Jinja2
API Gateway / Backend Layer	Handles incoming requests, validates inputs, manages API communication, and coordinates application workflows.	FastAPI, Python
Authentication Service	Manages user registration, login, session handling, and secure access control.	JWT Authentication, OAuth2
Story Generation Module	Generates comic storylines, dialogues, narrations, and panel descriptions using Large Language Models.	Google Gemini API
Image Generation Module	Produces comic panel illustrations based on generated scene descriptions and image prompts.	Hugging Face Inference API, FLUX
Comic Layout Engine	Combines generated text and images into a structured comic layout.	Python, Pillow
PDF Generation Service	Converts generated comics into downloadable PDF documents.	FPDF
Database Layer	Stores user data, comic metadata, generation history, and generated comic files.	PostgreSQL, SQLite
External AI Services	Provides story generation, prompt enhancement, and image generation capabilities.	Gemini API, Hugging Face API
Storage Repository	Stores generated comic images, PDF files, and application logs.	Local Storage, Cloud Storage

Data Flow Through Architecture

Step 1

User enters a comic prompt, selects an art style, and submits the request through the web interface.

Step 2

The FastAPI backend validates user input and routes the request to the appropriate service modules.

Step 3

The Story Generation Module communicates with Google Gemini API to generate:

- Comic storyline
- Dialogues
- Narration
- Panel descriptions

Step 4

The Image Generation Module sends scene descriptions to image-generation APIs to create comic illustrations.

Step 5

The Comic Layout Engine combines generated text and illustrations into a comic structure.

Step 6

The PDF Generation Service converts the comic into a downloadable PDF document.

Step 7

Generated comics, metadata, and user information are stored in the database and storage repository.

Step 8

The completed comic is displayed to the user for preview and download.

Security Considerations

- JWT-based user authentication.
 - Secure API communication using HTTPS.
 - Input validation and sanitization.
 - Role-based access control for administrators.
 - Secure storage of user information.
 - Protection against unauthorized API access.
-

Scalability Considerations

- Modular service architecture.
 - Cloud deployment support.
 - Containerized deployment using Docker.
 - Database scalability using PostgreSQL.
 - Independent scaling of AI services and backend APIs.
-

Advantages of the Architecture

- Easy maintenance and updates.
 - Scalable for future growth.
 - Supports multiple AI integrations.
 - Improved performance through modular design.
 - Secure user authentication and data handling.
 - Supports future features such as multi-language comics and mobile applications.
-

Conclusion

The ComicCraft Solution Architecture provides a scalable, secure, and modular framework for AI-powered comic generation. By integrating modern web technologies, FastAPI backend services, Generative AI models, image-generation systems, and cloud-ready storage solutions, the architecture enables users to transform simple story ideas into complete downloadable comics efficiently and effectively.